

GSOI 2026 暑期常规赛

第六试题解

Ehundategh & tfbz

Gioush 大队

2026 年 7 月

目录

T1 千仓平丰歉 (balance)

T2 迢迢寻故人 (trail)

T3 万山與共渡 (trans)

T4 万象启星环 (circum)

吐槽时间

Contents

T1 千仓平丰歉 (balance)

T2 迢迢寻故人 (trail)

T3 万山與共渡 (trans)

T4 万象启星环 (circum)

吐槽时间

数据点 1 ~ 4 20Pts

- 首先，看到这一档的数值范围很小，于是可以直接枚举最后所有仓库的最小值 L 与最大值 R ，再对每座仓库搜索它能否通过若干次操作落入区间 $[L, R]$ 。

数据点 1 ~ 4 20Pts

- ▶ 首先，看到这一档的数值范围很小，于是可以直接枚举最后所有仓库的最小值 L 与最大值 R ，再对每座仓库搜索它能否通过若干次操作落入区间 $[L, R]$ 。
- ▶ 第一个能够容纳每座仓库某个可达值的区间就是答案，这样做的时间复杂度为 $\mathcal{O}(nV^2)$ 。于是我们需要考虑的，就是怎样更快地刻画每座仓库所有可能到达的数值。

数据点 5 ~ 8 40Pts

- 根据上一档部分分的启发，考虑一次操作究竟会保留什么。不妨令 $d = \gcd(a, b)$ ，因为每次只会增加 a 或 b ，所以当前数值模 d 的余数始终不会改变。

数据点 5 ~ 8 40Pts

- ▶ 根据上一档部分分的启发，考虑一次操作究竟会保留什么。不妨令 $d = \gcd(a, b)$ ，因为每次只会增加 a 或 b ，所以当前数值模 d 的余数始终不会改变。
- ▶ 另一方面，根据 Bézout 定理，当增加量足够大时，同一个余数类中的整数都可以到达。特别地，当 $d = 1$ 时，可以把所有仓库都提升到同一个足够大的整数，于是答案恒为 0。

数据点 9 ~ 12 60Pts

- ▶ 于是原来的数值大小已经不再重要，只需要记录余数 $r_i = c_i \bmod d$ 。但是余数 0 与 $d - 1$ 实际上相邻，若仍然在普通数轴上寻找覆盖区间，就会漏掉跨过 0 的情况。

数据点 9 ~ 12 60Pts

- ▶ 于是原来的数值大小已经不再重要，只需要记录余数 $r_i = c_i \bmod d$ 。但是余数 0 与 $d-1$ 实际上相邻，若仍然在普通数轴上寻找覆盖区间，就会漏掉跨过 0 的情况。
- ▶ 因此可以把全部余数放在一个周长为 d 的圆周上。这一档有 $d \leq 100$ ，直接枚举圆弧的起点，再检查它能否覆盖全部余数即可，时间复杂度为 $\mathcal{O}(nd)$ 。

数据点 13 ~ 15 75Pts

- 接下来考虑 $a = b$ 的情况，此时 $d = a$ 。将余数排序为 $r_1 \leq r_2 \leq \dots \leq r_n$ ，若从圆周上删去一段不包含任何余数的空隙，那么剩下的圆弧就一定能够覆盖全部仓库。

数据点 13 ~ 15 75Pts

- ▶ 接下来考虑 $a = b$ 的情况，此时 $d = a$ 。将余数排序为 $r_1 \leq r_2 \leq \dots \leq r_n$ ，若从圆周上删去一段不包含任何余数的空隙，那么剩下的圆弧就一定能够覆盖全部仓库。
- ▶ 显然，为了让剩下的圆弧尽可能短，应当删去最长的空隙。除了相邻余数之差，还要考虑首尾之间的环形空隙 $r_1 + d - r_n$ ，排序后扫描一遍即可。

数据点 16 ~ 20 100Pts

- 对于一般情况，根据上一档部分分的方法，仍然令 $d = \gcd(a, b)$ ，再排序全部 $c_i \bmod d$ 。不妨令相邻余数以及首尾环形余数之间的最大空隙为 G ，于是最短覆盖圆弧长度为

$$\text{Ans} = d - G.$$

数据点 16 ~ 20 100Pts

- 对于一般情况，根据上一档部分分的方法，仍然令 $d = \gcd(a, b)$ ，再排序全部 $c_i \bmod d$ 。不妨令相邻余数以及首尾环形余数之间的最大空隙为 G ，于是最短覆盖圆弧长度为

$$\text{Ans} = d - G.$$

- 必要性来自每次操作都不会改变模 d 的余数，而充分性来自同一余数类中的数值最终都可以到达足够大的位置。因此直接维护最大环形空隙即可，总时间复杂度为 $\mathcal{O}(n \log n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 100Pts

```
1  int Gcd(int x,int y){
2      return y?Gcd(y,x%y):x;
3  }
4  void Solve(){
5      scanf("%d%d%d",&n,&a,&b);
6      int d=Gcd(a,b);
7      for(int i=1;i<=n;i++) scanf("%d",&c[i]),c[i]%d;
8      sort(c+1,c+n+1);
9      int Max=c[1]+d-c[n];
10     for(int i=2;i<=n;i++) Max=max(Max,c[i]-c[i-1]);
11     printf("%d\n",d-Max);
12 }
```

Contents

T1 千仓平丰歉 (balance)

T2 迢迢寻故人 (trail)

T3 万山與共渡 (trans)

T4 万象启星环 (circum)

吐槽时间

数据点 1 ~ 4 20Pts

- ▶ 首先，考虑直接枚举最后的答案 k 。暂时删去所有数值为 k 的格子，再用四联通 BFS 判断 $[1, 1]$ 与 $[n, m]$ 是否仍然连通，第一次能够绕开 k 时，答案就是 k 。

数据点 1 ~ 4 20Pts

- ▶ 首先，考虑直接枚举最后的答案 k 。暂时删去所有数值为 k 的格子，再用四联通 BFS 判断 $[1, 1]$ 与 $[n, m]$ 是否仍然连通，第一次能够绕开 k 时，答案就是 k 。
- ▶ 这是因为所有更小的数值都无法被绕开，而路线又允许重复经过格子，所以可以在同一个连通区域中依次访问它们。这样做的时间复杂度为 $\mathcal{O}(nm(n+m))$ 。

数据点 5 ~ 8 40Pts

- ▶ 接下来考虑这一档的特殊性质，同一行中的所有数都相等。从第 1 行走到第 n 行时，任意路线都必须经过每一行，于是每一行的代表值都一定会被经过。

数据点 5 ~ 8 40Pts

- ▶ 接下来考虑这一档的特殊性质，同一行中的所有数都相等。从第 1 行走到第 n 行时，任意路线都必须经过每一行，于是每一行的代表值都一定会被经过。
- ▶ 因此直接求所有行代表值形成的集合的 mex 即可，时间复杂度为 $\mathcal{O}(n)$ 。从这一档可以发现，当同值格子横贯整张网格时，它们会形成一条无法绕开的屏障。

数据点 9 ~ 12 60Pts

- ▶ 当 $\min(n, m) = 1$ 时，网格退化为一条线，从起点到终点必须经过所有格子，于是答案就是整条线数值集合的 mex，可以用布尔数组在线性时间内求出。

数据点 9 ~ 12 60Pts

- ▶ 当 $\min(n, m) = 1$ 时，网格退化为一条线，从起点到终点必须经过所有格子，于是答案就是整条线数值集合的 mex，可以用布尔数组在线性时间内求出。
- ▶ 换句话说，一个数值是否一定会出现，取决于这个数值对应的格子能否把起点与终点分隔开。于是正解需要做的，就是同时判断每一个较小的数值是否形成了这样的分隔。

数据点 13 ~ 15 75Pts

- ▶ 根据第一档部分分的方法，仍然逐个枚举 k 并做连通性判定即可。特别地，任取一条从左上到右下的最短路径，它只经过 $n + m - 1$ 个格子，所以答案一定不会超过 $n + m$ 。

数据点 13 ~ 15 75Pts

- ▶ 根据第一档部分分的方法，仍然逐个枚举 k 并做连通性判定即可。特别地，任取一条从左上到右下的最短路径，它只经过 $n + m - 1$ 个格子，所以答案一定不会超过 $n + m$ 。
- ▶ 因此枚举只需要进行到 $n + m$ ，这一档已经保留了正解中“判断某个数值能否绕开”的完整过程，只是还没有把不同数值的搜索合并起来。

正解 100Pts

- 现在固定一个数值 k ，把所有数值为 k 的格子记为 B_k ；把下边界与左边界合称为 A ，把上边界与右边界合称为 B 。

正解 100Pts

- ▶ 现在固定一个数值 k ，把所有数值为 k 的格子记为 B_k ；把下边界与左边界合称为 A ，把上边界与右边界合称为 B 。
- ▶ 若 B_k 中的一个八联通块同时碰到 A 与 B ，就称它为一堵**障碍墙**。我们要证明：起点能够绕开数值 k 到达终点，当且仅当不存在**障碍墙**。

正解 100Pts

- ▶ 先证明一个离散交叉结论。在所有连接 $[1, 1]$ 与 $[n, m]$ 的四联通格路中，取格子数最少的一条 P ，标记所有恰好与一个 P 中格子相邻的单位网格边。在每个格点处连接标记边，并约定所有能够共点移动的非 P 格子始终位于同一侧；出现四条标记边时，这一约定唯一确定其配对方式。

正解 100Pts

- ▶ 先证明一个离散交叉结论。在所有连接 $[1, 1]$ 与 $[n, m]$ 的四联通格路中，取格子数最少的一条 P ，标记所有恰好与一个 P 中格子相邻的单位网格边。在每个格点处连接标记边，并约定所有能够共点移动的非 P 格子始终位于同一侧；出现四条标记边时，这一约定唯一确定其配对方式。
- ▶ 这样得到的边界轨迹分别贴着格路 P 的两侧，并把边界集合 A 与 B 分开。八联通移动若不进入 P ，共边移动不可能穿过标记边，共点移动也会被刚才的配对规则留在同一侧。因此，任何连接 A 与 B 的八联通格链都必须与 P 共用一个格子。

正解 100Pts

- ▶ 先证明**障碍墙**确实能够挡住所有路径。反设删除 B_k 后仍存在四联通路径 P ，那么根据上一页的离散交叉结论， P 必须与这堵**障碍墙**共用一个格子，这与 P 不经过 B_k 矛盾。

正解 100Pts

- ▶ 先证明**障碍墙**确实能够挡住所有路径。反设删除 B_k 后仍存在四联通路径 P ，那么根据上一页的离散交叉结论， P 必须与这堵**障碍墙**共用一个格子，这与 P 不经过 B_k 矛盾。
- ▶ 再证明路径被挡住时一定存在**障碍墙**。若起点或终点本身属于 B_k ，包含该格子的八联通块已经同时碰到两组边界。否则，令 R 为删除 B_k 后从 $[1, 1]$ 能够到达的全部格子；因为 $[n, m]$ 不可达，所以它不属于 R 。

正解 100Pts

- 标记所有一侧属于 R 、另一侧不属于 R 的单位网格边。在每个格点处，按照使能够共点移动的非 R 格子位于同一侧的规则连接标记边。每个格点关联的标记边数只能为 0, 2, 4, 因此这些边组成若干闭合轨迹，或端点位于网格外框上的边界轨迹。

正解 100Pts

- ▶ 标记所有一侧属于 R 、另一侧不属于 R 的单位网格边。在每个格点处，按照使能够共点移动的非 R 格子位于同一侧的规则连接标记边。每个格点关联的标记边数只能为 0, 2, 4, 因此这些边组成若干闭合轨迹，或端点位于网格外框上的边界轨迹。
- ▶ 每一条不在外框上的标记边，其非 R 一侧都必须属于 B_k 。否则该侧是一个非 B_k 格子，又与 R 中格子共边，它也应该能够从 $[1, 1]$ 到达，这与它不属于 R 矛盾。

正解 100Pts

- 分别沿外框的 A 段与 B 段从 $[1, 1]$ 走向 $[n, m]$ 。相邻格子是否属于 R 的状态都从真变为假，所以两段外框上各有奇数个边界轨迹端点。同一段外框内部的配对会消去偶数个端点，因此至少存在一条边界轨迹，其两个端点分别位于 A 与 B 。

正解 100Pts

- ▶ 分别沿外框的 A 段与 B 段从 $[1, 1]$ 走向 $[n, m]$ 。相邻格子是否属于 R 的状态都从真变为假，所以两段外框上各有奇数个边界轨迹端点。同一段外框内部的配对会消去偶数个端点，因此至少存在一条边界轨迹，其两个端点分别位于 A 与 B 。
- ▶ 沿这条轨迹依次取每条内部标记边非 R 一侧的格子。根据上一页，这些格子都属于 B_k ；相邻标记边共用一个格点，所以对应格子相同、共边或共点，因而构成八联通格链。轨迹的两个端点分别位于 A 与 B ，这条格链正是一堵**障碍墙**，必要性得证。

正解 100Pts

- ▶ 接下来考虑怎样把所有数值的搜索合并起来。从四条边界出发，对数值相同的格子做八联通 DFS，并用参数记录搜索从哪一条边界开始；当连通块到达另一组边界时，就令该数值的标记为真。

正解 100Pts

- ▶ 接下来考虑怎样把所有数值的搜索合并起来。从四条边界出发，对数值相同的格子做八联通 DFS，并用参数记录搜索从哪一条边界开始；当连通块到达另一组边界时，就令该数值的标记为真。
- ▶ 由于 Visit 数组保证每个格子至多进入 DFS 一次，最后只需要从 0 开始找到第一个没有被标记的数值。时间复杂度为 $\mathcal{O}(nm)$ ，空间复杂度为 $\mathcal{O}(nm)$ 。

正解 100Pts

```
1 void DFS(int i,int j,int W) {  
2     if (Visit[i][j]) return;  
3     Visit[i][j]=1;  
4     if (i==1&&W!=3&&W!=2) Tag[Line[i][j]]=1;  
5     if (i==n&&W!=4&&W!=1) Tag[Line[i][j]]=1;  
6     if (j==1&&W!=1&&W!=4) Tag[Line[i][j]]=1;  
7     if (j==m&&W!=2&&W!=3) Tag[Line[i][j]]=1;  
8     if (Line[i+1][j]==Line[i][j]&&i+1<=n) DFS(i+1,j,W);  
9     if (Line[i-1][j]==Line[i][j]&&i-1>=1) DFS(i-1,j,W);  
10    if (Line[i][j+1]==Line[i][j]&&j+1<=m) DFS(i,j+1,W);  
11    if (Line[i][j-1]==Line[i][j]&&j-1>=1) DFS(i,j-1,W);
```

正解 100Pts

```
1  if (Line[i+1][j+1]==Line[i][j]&& i+1<=n&& j+1<=m) DFS(i+1,j+1,W);
2  if (Line[i-1][j+1]==Line[i][j]&& i-1>=1&& j+1<=m) DFS(i-1,j+1,W);
3  if (Line[i+1][j-1]==Line[i][j]&& i+1<=n&& j-1>=1) DFS(i+1,j-1,W);
4  if (Line[i-1][j-1]==Line[i][j]&& i-1>=1&& j-1>=1) DFS(i-1,j-1,W);
5  return;
6  }
```

正解 100Pts

```
1  for (int i=1;i<=n;i++) {  
2      if (Line[i][1]<=n+m) DFS(i,1,1);  
3      if (Line[i][m]<=n+m) DFS(i,m,2);  
4  }  
5  for (int i=1;i<=m;i++) {  
6      if (Line[1][i]<=n+m) DFS(1,i,3);  
7      if (Line[n][i]<=n+m) DFS(n,i,4);  
8  }  
9  for (int i=0;i<=n+m;i++) {  
10     if (!Tag[i]){ printf("%d\n",i); return;}  
11 }
```

Contents

T1 千仓平丰歉 (balance)

T2 迢迢寻故人 (trail)

T3 万山與共渡 (trans)

T4 万象启星环 (circum)

吐槽时间

数据点 1 ~ 4 20Pts

- ▶ 题目给定 $n \leq 100$ 且 $m = 1$ ，因此可以直接 DFS 一次，提取出唯一一批货物经过的链，并求出链长 L 。

数据点 1 ~ 4 20Pts

- ▶ 题目给定 $n \leq 100$ 且 $m = 1$, 因此可以直接 DFS 一次, 提取出唯一一批货物经过的链, 并求出链长 L 。
- ▶ 同时取这条链上最大的道路权值 $W_{\max}$ 。显然把它降为 0 最优, 答案就是 $L - W_{\max}$, 时间复杂度为 $\mathcal{O}(n)$ 。

数据点 5 ~ 8 40Pts

- ▶ 当 $m \neq 1$ 时，最大的问题是多批货物的运输路径可能重叠，不能再提取最长链并只看其中的最大边。

数据点 5 ~ 8 40Pts

- ▶ 当 $m \neq 1$ 时，最大的问题是多批货物的运输路径可能重叠，不能再提取最长链并只看其中的最大边。
- ▶ 由于这一档的 n, m 都不大，仍然尝试枚举：枚举把哪条道路降为 0，再重新计算全部运输路径并取最大值。时间复杂度为 $\mathcal{O}(nm \log n)$ 。

数据点 9 ~ 14 70Pts

- ▶ 现在来看链的情况。注意到题目要求最小化最大值，可以二分一个时间 t 。原本花费不超过 t 的货物无需处理；其余货物都必须经过钦定的 0 时间道路。

数据点 9 ~ 14 70Pts

- ▶ 现在来看链的情况。注意到题目要求最小化最大值，可以二分一个时间 t 。原本花费不超过 t 的货物无需处理；其余货物都必须经过钦定的 0 时间道路。
- ▶ 换句话说，这条道路一定被所有原本时间大于 t 的运输路径覆盖。问题便转化成链上的区间覆盖次数，可以用差分与前缀和维护。

数据点 9 ~ 14 70Pts

- ▶ 设时间大于 t 的路径共有 c 条。若一条道路被覆盖了 c 次，它就是可选道路；所有可选道路中权值最大的道路一定最优。

数据点 9 ~ 14 70Pts

- ▶ 设时间大于 t 的路径共有 c 条。若一条道路被覆盖了 c 次，它就是可选道路；所有可选道路中权值最大的道路一定最优。
- ▶ 设该权值为 w ，最长运输路径为 $L_{\max}$ 。若 $L_{\max} - w \leq t$ ，则当前 t 合法。一次判断为 $\mathcal{O}(n + m)$ ，总复杂度为 $\mathcal{O}((n + m) \log V)$ 。

正解 100Pts

- 考虑将上一档做法推广到树上。仍然二分答案，只需把链上的区间覆盖改成树上差分。对于一条超时路径 (u, v) ，令 $z = \text{lca}(u, v)$ ，作

$$\begin{cases} d_u \leftarrow d_u + 1, \\ d_v \leftarrow d_v + 1, \\ d_z \leftarrow d_z - 2. \end{cases}$$

正解 100Pts

- 考虑将上一档做法推广到树上。仍然二分答案，只需把链上的区间覆盖改成树上差分。对于一条超时路径 (u, v) ，令 $z = \text{lca}(u, v)$ ，作

$$\begin{cases} d_u \leftarrow d_u + 1, \\ d_v \leftarrow d_v + 1, \\ d_z \leftarrow d_z - 2. \end{cases}$$

- 树上差分统计的是点，因此把道路 $(u, \text{fa}(u))$ 看作由结点 u 派生。DFS 回溯时令 $d_u \leftarrow d_u + \sum d_v$ ，此时 d_u 就是该道路被多少条超时路径覆盖。

正解 100Pts

- 若某条道路的覆盖次数等于超时路径总数 c ，它便同时属于全部超时路径。设所有这种道路的最大权值为 w ，最长运输路径长度为 $L_{\max}$ ，判定条件为

$$L_{\max} - w \leq t.$$

正解 100Pts

- ▶ 若某条道路的覆盖次数等于超时路径总数 c ，它便同时属于全部超时路径。设所有这种道路的最大权值为 w ，最长运输路径长度为 $L_{\max}$ ，判定条件为

$$L_{\max} - w \leq t.$$

- ▶ 路径长度和 LCA 预处理为 $\mathcal{O}((n+m)\log n)$ ，每次判定为 $\mathcal{O}(n+m)$ ，总时间复杂度为 $\mathcal{O}((n+m)\log V)$ 。

正解 100Pts

```
1 bool Judge(int t){
2     int cnt=0;
3     Tag=0;
4     memset(Diff,0,sizeof(Diff));
5     for(int i=1;i<=m;i++){
6         if(p[i].Val<=t) continue;
7         Diff[p[i].u]++;Diff[p[i].v]++;
8         Diff[p[i].lca]-=2;
9         cnt++;
10    }
11    Run(1,1,cnt,p[m].Val-t);
12    if(Tag) return 1;
13    else return 0;
14 }
```

正解 100Pts

```
1 inline int Run(int Now,int From,int cnt,int t){
2     int Sum=0,tmp=0;
3     for(int i=Head[Now];i;i=Edge[i].Next){
4         int To=Edge[i].Ed;
5         if(To==From)continue;
6         tmp=Run(To,Now,cnt,t);
7         Sum+=tmp;
8         if(tmp==cnt&&Edge[i].Val>=t){Tag=1;}
9     }
10    return Sum+Diff[Now];
11 }
```

Contents

T1 千仓平丰歉 (balance)

T2 迢迢寻故人 (trail)

T3 万山與共渡 (trans)

T4 万象启星环 (circum)

吐槽时间

数据点 1 ~ 4 20Pts

- ▶ 首先，看到这个题目，是一个计数题，于是我们考虑 DP，但是直接统计包含至少一个质数的方案数不好统计，于是我们将其转化为 – 任意被 p 整除的情况-不含质数且被 p 整除的情况。

数据点 1 ~ 4 20Pts

- ▶ 首先，看到这个题目，是一个计数题，于是我们考虑 DP，但是直接统计包含至少一个质数的方案数不好统计，于是我们将其转化为 - 任意被 p 整除的情况-不含质数且被 p 整除的情况。
- ▶ 于是我们分开考虑，可以料想，当前模 p 的余数 r 也是我们转移的依据，不妨令 $f_{i,r,0/1}$ 表示已经决定好前 i 个数，且余数当前为 r 的方案数，第三位表示可含/不含质数。

数据点 1 ~ 4 20Pts

于是我们得到状态转移方程如下：

$$\begin{cases} f_{i,r,0} \leftarrow \sum_{j=1}^m f_{i-1,(r-j+p) \bmod p,0} \\ f_{i,r,0} \leftarrow \sum_{\substack{j=1 \\ j \text{ is not prime}}}^m f_{i-1,(r-j+p) \bmod p,0} \end{cases}$$

时间复杂度 $\mathcal{O}(nmp)$ 。

数据点 5 ~ 8 20Pts

- ▶ 上述的算法虽然直接，但是时间复杂度太高了，仔细观察可以发现，在滚动数组后，我们对任意的 i 执行的转移都是若干次相同的转移，于是这就启发我们用矩阵维护这样的转移。

数据点 5 ~ 8 20Pts

- ▶ 上述的算法虽然直接，但是时间复杂度太高了，仔细观察可以发现，在滚动数组后，我们对任意的 i 执行的转移都是若干次相同的转移，于是这就启发我们用矩阵维护这样的转移。
- ▶ 令列向量 $\mathbf{x}_i = (f_{i,0,0} \ f_{i,1,0} \ \cdots \ f_{i,p-1,0})^T$ ，考虑找到一个矩阵 A 使得，对于任意 i ，都有 $\mathbf{x}_i = A\mathbf{x}_{i-1}$ ，这样就可以转化成 $\mathbf{x}_n = A^n \mathbf{x}_0$ 。

数据点 5 ~ 8 40Pts

- 接下来考虑怎么找到这样的矩阵 A ，根据矩阵乘法的定义，把 Ax_{i-1} 直接展开，对于每一行，我们得到恒等式 $f_{i,j,0} = \sum_{k=1}^m a_{j,k} f_{i-1,k,0}$ ，将这个式子与 DP 方程直接对比，于是我们得到 $a_{j,k}$ 的值应该为满足 $x + k \equiv j \pmod{p}$ 且 $1 \leq x \leq m$ 的 x 的数量。

数据点 5 ~ 8 40Pts

- ▶ 接下来考虑怎么找到这样的矩阵 A ，根据矩阵乘法的定义，把 Ax_{i-1} 直接展开，对于每一行，我们得到恒等式
$$f_{i,j,0} = \sum_{k=1}^m a_{j,k} f_{i-1,k,0}$$
，将这个式子与 DP 方程直接对比，于是我们得到 $a_{j,k}$ 的值应该为满足 $x + k \equiv j \pmod{p}$ 且 $1 \leq x \leq m$ 的 x 的数量。
- ▶ 不妨记 $\text{Count}(i, 0)$ 为同余于 i 的 x 的个数，那么我们需要的值就是 $\text{Count}((j - k + p) \bmod p)$ 的值。对应的，我们也可以据此记录 $\text{Count}(i, 1)$ 为同余于 i 且不是质数的 x 的个数。

数据点 5 ~ 8 40Pts

于是可以得到转移矩阵：

$$A = \begin{pmatrix} \text{Count}(0,0) & \text{Count}(p-1,0) & \cdots & \text{Count}(1,0) \\ \text{Count}(1,0) & \text{Count}(0,0) & \cdots & \text{Count}(2,0) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Count}(p-1,0) & \text{Count}(p-2,0) & \cdots & \text{Count}(0,0) \end{pmatrix}.$$

同理，把 $\text{Count}(i,0)$ 换成 $\text{Count}(i,1)$ 就得到了另一个转移矩阵，不妨将两个矩阵分别记为 f, g ，于是令 $y_0 = (1, 0, 0, \dots, 0)$ ，则 $y_n = (f^n - g^n)y_0$ ，可以用矩阵快速幂维护，加上暴力求解 $\text{Count}(i,j)$ ，时间复杂度是 $\mathcal{O}(p^3 \log n + m\sqrt{m})$ 。

正解 100Pts

直接根据上一档的方法，瓶颈在于计算 $\text{Count}(i, j)$ ，对于 $j = 0$ 的情况，直接用带余除法处理即可，而对于 $j = 1$ 的情况，我们可以在带余除法处理后，用筛法筛出所有的质数，再从 $j = 0$ 的情况中减即可，时间复杂度 $\mathcal{O}(m \log \log m + p^3 \log n)$ ，但显然若带 $\log$ 套 $\log$ 必然过不了，所以用线性筛法，于是时间复杂度 $\mathcal{O}(m + p^3 \log n)$ 。

Contents

T1 千仓平丰歉 (balance)

T2 迢迢寻故人 (trail)

T3 万山與共渡 (trans)

T4 万象启星环 (circum)

吐槽时间